

When Simulators Disagree

A guide to the project for co-authors — what we asked, why it mattered, what we did, and what we found.

Khushi Jaiswal · COEP Technological University · August 2026

The one-paragraph version. If you train a walking robot in a simulator, you want to know it will work in the real world. Most people cannot test on a real robot, so they use a substitute: train in one simulator, test in a second one, and if the robot still walks, call it validated. We asked whether that substitute test actually proves anything. It mostly does not. We found the simulators genuinely disagree with each other, that the disagreement comes from the software rather than from rounding errors, and yet a trained robot barely notices. So passing the test tells you much less than people assume.

1. What problem were we looking at?

Robots that walk are usually trained in simulation. Training on real hardware is slow, expensive, and breaks the robot. So you train in a physics simulator where the robot can fall over a million times for free.

The catch is that simulators are not reality. A policy that walks beautifully in simulation may fall over immediately on a real robot. This is called the **reality gap**.

Testing on hardware is the honest way to check. Most research groups, including ours, do not have a humanoid robot. So the field uses a stand-in called **sim-to-sim validation**:

Sim-to-sim validation, in plain terms. Train your robot in Simulator A. Then, without changing anything, run it in Simulator B. If it still walks well, that is taken as evidence it will survive the real world too.

The reasoning: if the controller is not fragile enough to break when the physics changes slightly, it is probably not fragile enough to break on hardware either.

This is done constantly and is rarely questioned.

2. The research gap: what nobody had checked

Here is the thing that bothered us. Everyone uses this test. Nobody had measured whether the test is any good.

Specifically, three questions had no published answer:

1. **How different are these simulators, really?** People say "we validated across simulators" without saying how far apart the two simulators actually are. If they barely differ, the test is trivial to pass.
2. **Why do they differ?** If it is just rounding error from using smaller numbers on a GPU, that will disappear as hardware improves. If the two programs actually disagree about physics, it will not.
3. **Does a robot even notice?** If the difference is smaller than the random variation you get from just retraining, then passing the test means nothing at all.

There is also a hidden problem. The test has a setting nobody reports: *which physics configuration* you ran the check under. Things like how soft you make the ground contact. If the answer changes depending on that setting, then "we validated in sim-to-sim" is not even a well-defined claim, and two papers saying it may be saying completely different things.

3. What we actually studied

MuJoCo is the physics simulator most legged-robot research uses. It now exists in three versions, and this is the key point — **they are supposed to be the same simulator**:

Version	What it is	Runs on	Number precision
MuJoCo-C	The original. The reference everyone trusts.	CPU	float64 (high)
MJX-JAX	A rewrite in JAX for GPU speed.	GPU	float32 (lower)
MuJoCo Warp	A rewrite in CUDA for GPU speed.	GPU	float32 (lower)

People swap between these freely, assuming they are interchangeable. That assumption is exactly what we tested.

We used the four humanoid robots that ship with MuJoCo Playground:

Robot	Motors	Mass	Height
Unitree G1	29	33.34 kg	0.785 m
Booster T1	23	31.61 kg	0.665 m
ROBOTIS Op3	20	3.15 kg	0.244 m
Berkeley Humanoid	12	16.06 kg	0.515 m

These are not all alike. Op3 is a 3 kg desktop toy; G1 is a 33 kg full-size humanoid. Ten times the mass, twice the motors.

How the measurement works

Simple idea. We build one robot model, put it in the exact same starting pose, give it the exact same motor commands, and run it in all three simulators at once. After one second of simulated time we measure how far apart the three robots have drifted.

Because everything else is identical, any difference must come from the simulator itself. Nothing else is left to blame.

4. The experiments, one at a time

Experiment 0 — Are the simulators even correct?

Question: before comparing simulators to each other, are any of them right?

Method: we used four physics problems where the answer can be worked out on paper — a dropped ball, a swinging pendulum, a block sliding down a slope, and a block that should sit still on a slope.

Result: with no contact, all three agree to the limit of float32 precision, and the two GPU versions are *identical*. Once contact is involved, they separate.

Why this matters: it proves that any bigger disagreement we find later comes from contact and the physics solver, not from rounding. It sets the floor for everything else.

A bonus finding that affects everyone: in the "block should not move" test, *every* simulator lets it creep about 1.1 mm per second. That is MuJoCo's contact model itself, not any one version. For a walking robot that is a planted foot sliding about a centimetre over ten seconds.

Experiment 1 — How far apart are they on a real humanoid?

Method: the G1 humanoid, held in a standing pose, changing one physics setting at a time.

Result: Warp is about 8× closer to the trusted C reference than JAX is. The revealing detail is that JAX-vs-Warp is the same size as C-vs-JAX. So Warp and C sit together, and **JAX is the odd one out** — not simply "CPU differs from GPU."

Also: contact softness controls the size. Stiff contacts 2.1×, default 8.1×, soft contacts 28.2×. So the answer genuinely depends on that unreported setting we worried about in Section 2.

Experiment 1b — Does this hold on other robots?

We repeated everything on all four robots across 14 physics settings, giving 56 combinations.

Result: 34 of 56 show a real difference after correcting for multiple testing, ranging from 2× to 82×.

An important lesson for how people test. We first compared the robots only at their default settings — which is exactly what one sim-to-sim check does — and concluded "only G1 is affected." The full grid showed that was wrong. Testing at one configuration gives a misleading answer.

Experiment 1c — The Op3 surprise

One robot, Op3, looked immune: only 1 of 14 settings showed anything. We nearly wrote that up as "some robots are unaffected."

Then we noticed Op3 ships with *coarser* simulation settings than the others (a bigger time step, and one solver iteration instead of three). We matched its settings to the other robots and changed nothing else.

1 of 14 became 13 of 14. The biggest effect was 63×. Op3 was never immune. Its default settings were simply too coarse to detect the problem.

This is arguably our most practically useful finding: **a sim-to-sim check can pass simply because your settings are too rough to notice the failure.**

Experiments 6 and 7 — Why do they differ?

This is the core of the paper. There were three possible explanations, and we ruled out two of them.

Could it be rounding? GPUs use lower-precision numbers. So we re-ran JAX using high precision.

- The no-contact difference vanished — it shrank by a factor of a billion. This proves the change worked.
- The contact difference did not move at all (ratio 1.05).

The first bullet is what makes the second one convincing. Without it, "high precision did not help" could just mean we did it wrong.

Could it be step size? Simulators advance in small time steps. Errors from that shrink as you use smaller steps. So we made the steps 8× smaller.

- Warp's disagreement shrank steadily toward zero, exactly as a step-size error should.
- JAX's disagreement stopped shrinking and flattened out. Smaller steps did not help.

Conclusion. Two independent tests, same answer. Warp behaves like the same physics computed slightly differently. JAX behaves like *different physics*. The disagreement is in the software's logic, not in its arithmetic.

This matters for the future: a rounding problem would fade as hardware improves. A logic difference will not. It stays until someone changes the code.

Experiments 2 and 5 — Does a trained robot notice?

Now the question a robotics audience actually cares about. We trained three walking policies and tested each one in the different simulators.

The crucial part is what we compare against. A difference only counts if it is bigger than the noise you get from just retraining with a different random seed. So we measured that noise from the same runs.

Comparison	Difference found	Verdict
JAX vs Warp	$t = 1.400$	0.56× seed noise — nothing
JAX vs C reference	2.7% worse, $t = 2.877$	0.53× seed noise — real but irrelevant

Two things are true at once. There *is* a consistent, measurable effect. And it is *smaller than simply retraining the robot*. We report it as no effect, using a rule we fixed before looking at the results.

A related detail worth knowing: the robots' actual paths diverge a lot — 27 cm apart after ten seconds — while their scores stay the same. So any claim resting on one specific run (a demo video, a particular failure) is simulator-dependent, even though the averages are not.

Experiment 3 — How much difference can a robot tolerate?

The deeper question: performance must drop as two simulators get further apart. Where is the breaking point?

We deliberately broke the physics by known amounts and measured the damage. The surprise was that the amount of difference is not what matters.

What we changed	Damage at the same measured difference
Friction (ground contact)	51.7% worse
Armature (joint inertia)	1.3% worse

A 40× difference in damage, at identical measured divergence. So *where* two simulators differ matters far more than *how much* they differ.

This makes sense physically: the robot's controller can compensate for heavier joints, but it cannot compensate for the ground behaving differently under its feet.

It also explains the null result above. A single number summarising "how different are these simulators" cannot distinguish a harmless mismatch from a damaging one — which is exactly what a naive sim-to-sim check reports.

Experiment 4 — Testing our own best guess, and being wrong

G1 showed the biggest effect and was also the only robot describing foot contact in a particular way in its model file. Tempting explanation.

So we tested it: we rewrote the model to describe the identical contact the other way, and confirmed the two versions are *bit-for-bit identical* under MuJoCo-C, meaning the change is genuinely cosmetic.

Result: the effect survived unchanged. Our hypothesis was wrong. We include it because ruling out an explanation is worth more than leaving it hanging.

5. How we avoided fooling ourselves

This section matters more than it sounds. **Four times during this project we got a confident, plausible, completely wrong answer.** No crash, no error message, just a wrong number that looked fine.

The mistake	What it told us	The truth
Compared runs of equal <i>steps</i> instead of equal <i>time</i>	Time step is the dominant factor, 109×	No effect at all
Used averages on very skewed data	One robot's effect was 3.32×	2.91×, once measured properly
Compared two quantities with different units	A 3.17× effect	The data contained no effect
Simulator silently dropped contacts	A full set of Op3 results	Physically meaningless; all discarded

Each now has a guard that makes it fail loudly instead of quietly. The last one is the scariest: the simulator was printing a warning to a place nothing was reading, and returning wrong physics as if all was well.

One thing to know before you run anything. Results on GPU are not exactly repeatable. Run the identical evaluation twice and you get answers differing by about 1.86%. If you see a small difference from a number in the paper, that may be why — not a bug. Any improvement smaller than that is indistinguishable from simply running again.

6. What we conclude

1. **The three MuJoCo versions are not interchangeable once contact is involved.** 34 of 56 tests across four robots, differences of $2\times$ to $82\times$.
2. **The cause is the software's logic, not rounding or step size.** Two independent tests each rule out one explanation while the difference survives.
3. **A trained robot absorbs it anyway.** The 2.7% drop is real but smaller than the effect of retraining with a different seed.
4. **Where simulators differ matters more than how much.** Contact errors are $\sim 40\times$ more damaging than joint-inertia errors of the same size.
5. **Therefore sim-to-sim validation is a weak test.** It probes a difference below the level a robot responds to. Passing it is not strong evidence of anything.

What we suggest people do instead: report the physics configuration the check was run under, compare any difference against retraining noise from the same runs, and be suspicious of a clean pass at default settings — remember Op3.

7. What we cannot claim

Being honest about limits is part of the paper, and reviewers will look for this.

- **No hardware.** We can say which simulators disagree and by how much. We cannot say which one is closest to reality. This bounds everything above.
- **One task.** The four-robot comparison is broad, but all the robot-learning results are flat-ground walking on G1.
- **Limited compute.** A 4 GB laptop GPU, so policies are trained to about a quarter of the usual budget.
- **Mechanism unknown.** We show the difference is in the software's logic, and we ruled out one explanation, but we have not found which part of the solver differs.

8. Where things stand, and how to start

All experiments are finished. The paper is written in IEEE conference format and every number in it has been checked automatically against the raw data files.

Still to do: confirm the reference details in the bibliography, trim to the page limit for the target conference, and decide where to submit.

To get started:

- Repository: `github.com/KhushiJ2911/Biped-Locomotion-via-RL` — the README covers layout and findings.
- The paper is in `paper/ieee/`. The longer working draft, with reasoning the paper leaves out, is `paper/draft.md`.
- Raw data is in `results/`. Every number traces back to a file there.
- Run `python3 paper/ieee/verify_tex_numbers.py` after changing any number in the paper. It re-reads all 84 of them from the raw data.

Every figure and number in this document comes from a script in the repository and has been checked against its source data file.